Lab 7: Fuzz Testing for Input Validation Bugs

Software Engineering Lab · SRN PES1UG24CS068 · L6_L7_10 Bus Transit Fare

Tools: Python 3.13.1, Hypothesis 6.170.15, pytest 8.3.4. Application under test: `processor.py` (parse_transit_corridor, lookup_concession_discount, compute_passenger_density).

Bugs found by fuzzing and how they were fixed

Function	Failing input	Original behaviour	Fix
parse_transit_corridor	None	AttributeError on .strip()	Return None for non-string input
	" , 'DT-UP'	ValueError (tuple unpacking)	Require exactly 3 '-' separated parts
	'A-B-x', 'A-B-0', huge zones	ValueError / bad fare / OverflowError	Zones must be ASCII digits, 1 to 100; codes alphanumeric
lookup_concession_discount	None	AttributeError on .strip()	Return None for non-string input
	'', 'VIP'	KeyError	dict.get() returns None for unknown categories
compute_passenger_density	None	AttributeError on .split()	Return None for non-string input
	'', '/', '0/'	ValueError (unpack / float())	Check part count, catch ValueError
	'0/0'	ZeroDivisionError	Reject km <= 0
	'1/-1', 'nan/1', '1e308/1e-308'	Negative / nan / inf density	Reject negatives and non-finite values

Each screenshot below is the terminal output captured from the actual run (full logs are in the repo under `Lab-7/screenshots/`).

1. Execution of the original application

Sample input works; edge inputs crash with ValueError, KeyError and ZeroDivisionError.

```
Windows PowerShell - python processor.py (original)

$ python processor.py   (valid input: DT-UP-3 / STUDENT / 50/10.0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Multiplier: 0.5

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): Density: 5.0

$ python processor.py   (edge input: DT-UP / VIP / 50/0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Traceback (most recent call last):
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 28, in main
    print(parse_transit_corridor(c_input))
          ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 2, in parse_transit_corri
    dor
    orig, dest, zones_str = corridor_code.strip().split('-')
    ^^^^^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 3, got 2)

$ python processor.py   (edge input: DT-UP-3 / VIP / 50/0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Traceback (most recent call last):
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 32, in main
    print("Multiplier:", lookup_concession_discount(cat_input))
                         ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 16, in lookup_concession_
    discount
    return discounts[category]
           ~~~~~~~~~^^^^^^^^^^
KeyError: 'VIP'

$ python processor.py   (edge input: DT-UP-3 / STUDENT / 50/0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Multiplier: 0.5

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): Traceback (most recent call last):
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 36, in main
    print("Density:", compute_passenger_density(d_input))
                       ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7\processor.py", line 22, in compute_passenger_
    density
    density = passengers / km
              ~~~~~~~~~~~^~~~
ZeroDivisionError: float division by zero
```

2. Failing test cases found by fuzz testing (original processor.py)

5 of 8 Hypothesis tests fail against the original code.

```
========================================== short test summary info ==========================================
FAILED test_processor.py::test_parse_transit_corridor_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures.
(2 sub-exceptions)
FAILED test_processor.py::test_parse_transit_corridor_structured_no_crash - ExceptionGroup: Hypothesis found 2 distinct
 failures. (2 sub-exceptions)
FAILED test_processor.py::test_lookup_concession_discount_safe - ExceptionGroup: Hypothesis found 2 distinct failures.
(2 sub-exceptions)
FAILED test_processor.py::test_compute_passenger_density_safe - ExceptionGroup: Hypothesis found 3 distinct failures. (
3 sub-exceptions)
FAILED test_processor.py::test_compute_passenger_density_structured_safe - ExceptionGroup: Hypothesis found 5 distinct
failures. (5 sub-exceptions)
========================================== 5 failed, 3 passed in 2.46s ==========================================
(base) PS C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7>
```

Windows PowerShell - pytest output (falsifying examples)

```
Falsifying inputs reported by Hypothesis (excerpt of the same pytest run):

test_parse_transit_corridor_no_crash:
    FAILED -> parse_transit_corridor crashed on '' with: ValueError: not enough values to unpack (expected 3, got 1)
    FAILED -> parse_transit_corridor crashed on None with: AttributeError: 'NoneType' object has no attribute 'strip'

test_parse_transit_corridor_structured_no_crash:
    FAILED -> parse_transit_corridor crashed on '' with: ValueError: not enough values to unpack (expected 3, got 1)

test_lookup_concession_discount_safe:
    FAILED -> lookup_concession_discount crashed on '' with: KeyError: ''
    FAILED -> lookup_concession_discount crashed on None with: AttributeError: 'NoneType' object has no attribute 'strip'

test_compute_passenger_density_safe:
    FAILED -> compute_passenger_density crashed on '/' with: ValueError: could not convert string to float: ''
    FAILED -> compute_passenger_density crashed on '' with: ValueError: not enough values to unpack (expected 2, got 1)
    FAILED -> compute_passenger_density crashed on None with: AttributeError: 'NoneType' object has no attribute 'split'

test_compute_passenger_density_structured_safe:
    FAILED -> invalid density -1.0 for input '1/-1'
    FAILED -> compute_passenger_density crashed on '0/0' with: ZeroDivisionError: float division by zero
    FAILED -> compute_passenger_density crashed on '0/' with: ValueError: could not convert string to float: ''
    FAILED -> compute_passenger_density crashed on '/' with: ValueError: could not convert string to float: ''
    FAILED -> compute_passenger_density crashed on '' with: ValueError: not enough values to unpack (expected 2, got 1)
```

3. Execution of the corrected application

Valid input gives the same results as before; invalid input now prints a clear message instead of crashing.

```
Windows PowerShell - python processor.py (fixed)

$ python processor.py   (input: DT-UP-3 STUDENT 50/10.0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Multiplier: 0.5

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): Density: 5.0

$ python processor.py   (input: DT-UP VIP 50/0)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Invalid corridor code. Expected format ORIG-DEST-ZONES, e.g. DT-UP-3

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Unknown concession category. Valid: STANDARD, STUDENT, SENIOR, MILITARY, CHI
    LD

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): Invalid input. Expected PASSENGERS/KM with passengers >= 0 and km > 0, e.g. 50/10.0

$ python processor.py   (input: DT-UP-3 senior 1/-1)
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): Multiplier: 0.35

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): Invalid input. Expected PASSENGERS/KM with passengers >= 0 and km > 0, e.g. 50/10.0
```

4. Final test execution (fixed processor.py)

All 8 Hypothesis tests pass.

```
(base) PS C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7> python -m pytest test_processor.py
-v
============================================= test session starts =============================================
platform win32 -- Python 3.14.6, pytest-9.2.0, pluggy-1.6.0 -- C:\Users\annam\miniconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\annam\Documents\PESU\SEM 5\SE\LAB\7\SE-Labs-PES1UG24CS068\Lab-7
plugins: anyio-4.12.1, hypothesis-6.170.15
collected 8 items

test_processor.py::test_parse_transit_corridor_no_crash PASSED                                          [ 12%]
test_processor.py::test_parse_transit_corridor_structured_no_crash PASSED                               [ 25%]
test_processor.py::test_parse_transit_corridor_valid PASSED                                             [ 37%]
test_processor.py::test_lookup_concession_discount_safe PASSED                                          [ 50%]
test_processor.py::test_lookup_concession_discount_valid PASSED                                         [ 62%]
test_processor.py::test_compute_passenger_density_safe PASSED                                           [ 75%]
test_processor.py::test_compute_passenger_density_structured_safe PASSED                                [ 87%]
test_processor.py::test_compute_passenger_density_valid PASSED                                          [100%]

============================================== 8 passed in 2.66s ==============================================
```

5. Source code: processor.py (fixed)

```
import math

MAX_ZONES = 100

CONCESSION_DISCOUNTS = {
    "STANDARD": 1.00,
    "STUDENT": 0.50,
    "SENIOR": 0.35,
    "MILITARY": 0.20,
    "CHILD": 0.00
}


def parse_transit_corridor(corridor_code):
    """Parse 'ORIG-DEST-ZONES'. Returns a description string, or None if invalid."""
    # Bug fix: None / non-string input crashed on .strip()
    if not isinstance(corridor_code, str):
        return None

    parts = corridor_code.strip().split('-')
    # Bug fix: wrong number of '-' separated parts crashed the tuple unpacking
    if len(parts) != 3:
        return None
    orig, dest, zones_str = (p.strip() for p in parts)

    # Bug fix: empty / non-alphanumeric station codes were accepted
    if not (orig.isascii() and orig.isalnum() and dest.isascii() and dest.isalnum()):
        return None

    # Bug fix: non-numeric zones crashed int(); only plain ASCII digits allowed
    if not (zones_str.isascii() and zones_str.isdigit()):
        return None
    zones = int(zones_str)

    # Bug fix: zero zones and huge zone counts (float overflow) were accepted
    if zones < 1 or zones > MAX_ZONES:
        return None

    base_fare = 2.50 + (zones * 0.75)
    return f"Route {orig.upper()}->{dest.upper()}: {zones} Zones, Fare ${base_fare:.2f}"


def lookup_concession_discount(category_name):
    """Return the fare multiplier for a category, or None if the category is unknown."""
    # Bug fix: None / non-string input crashed on .strip()
    if not isinstance(category_name, str):
        return None
    category = category_name.strip()
    # Bug fix: non-ASCII look-alikes (e.g. '■tudent'.upper() == 'STUDENT') are rejected
    if not category.isascii():
        return None
    # Bug fix: unknown category raised KeyError
    return CONCESSION_DISCOUNTS.get(category.upper())


def compute_passenger_density(occupancy_ratio_str):
    """Compute passengers per km from 'PASSENGERS/KM'. Returns a float, or None if invalid."""
    # Bug fix: None / non-string input crashed on .split()
    if not isinstance(occupancy_ratio_str, str):
        return None

    parts = occupancy_ratio_str.split('/')
    # Bug fix: missing or extra '/' crashed the tuple unpacking
    if len(parts) != 2:
        return None

    # Bug fix: empty or non-numeric values crashed float()
    try:
        passengers = float(parts[0])
        km = float(parts[1])
    except ValueError:
        return None

    # Bug fix: nan / inf were accepted
    if not (math.isfinite(passengers) and math.isfinite(km)):
        return None
    # Bug fix: km == 0 raised ZeroDivisionError; negative values gave negative density
    if passengers < 0 or km <= 0:
```

```
        return None

    density = passengers / km
    # Bug fix: very large / very small values overflowed to inf
    if not math.isfinite(density):
        return None
    return round(density, 2)


def main():
    print("==== Running parse_transit_corridor ====")
    c_input = input("Enter corridor (e.g. DT-UP-3): ")
    result = parse_transit_corridor(c_input)
    print(result if result is not None else "Invalid corridor code. Expected format ORIG-DEST-ZONES, e.g. DT-UP-3")

    print("\n==== Running lookup_concession_discount ====")
    cat_input = input("Enter concession category (e.g. STUDENT, SENIOR): ")
    multiplier = lookup_concession_discount(cat_input)
    if multiplier is not None:
        print("Multiplier:", multiplier)
    else:
        print("Unknown concession category. Valid:", ", ".join(CONCESSION_DISCOUNTS))

    print("\n==== Running compute_passenger_density ====")
    d_input = input("Enter passengers/km (e.g. 50/10.0): ")
    density = compute_passenger_density(d_input)
    if density is not None:
        print("Density:", density)
    else:
        print("Invalid input. Expected PASSENGERS/KM with passengers >= 0 and km > 0, e.g. 50/10.0")


if __name__ == "__main__":
    main()
```

6. Source code: test_processor.py

```
"""
Lab 7 - Fuzz testing processor.py with Hypothesis.

Each function gets:
  1. A "no crash" fuzz test: random text / None must never raise an exception.
  2. A structured fuzz test: inputs shaped like real data (e.g. "AB-CD-3",
     "50/10.0") so Hypothesis reaches the deeper code paths.
  3. A property test: valid inputs must still give the correct answer.

Contract of the hardened processor: on invalid input a function returns None
instead of raising.
"""
import math

from hypothesis import given, settings, strategies as st
from processor import parse_transit_corridor, lookup_concession_discount, compute_passenger_density

VALID_CATEGORIES = {
    "STANDARD": 1.00,
    "STUDENT": 0.50,
    "SENIOR": 0.35,
    "MILITARY": 0.20,
    "CHILD": 0.00,
}

# Strings made of the characters each parser cares about, so the fuzzer
# hits separators, signs, digits and special float words often.
corridor_like = st.text(alphabet="AB09-+ .\t_x", max_size=15)
ratio_like = (
    st.text(alphabet="0123456789./-+eE ", max_size=15)
    | st.tuples(
        st.sampled_from(["50", "-5", "0", "nan", "inf", "-inf", "1e308", "", "abc"]),
        st.sampled_from(["10.0", "0", "-0.0", "nan", "inf", "1e-308", "", "x"]),
    ).map(lambda t: f"{t[0]}/{t[1]}")
)
codes = st.text(alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789", min_size=1, max_size=5)


# ---------------------------------------------------------------------------
# parse_transit_corridor
# ---------------------------------------------------------------------------
@given(st.text() | st.none())
def test_parse_transit_corridor_no_crash(s):
    try:
        result = parse_transit_corridor(s)
    except Exception as e:
        assert False, f"parse_transit_corridor crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or isinstance(result, str)


@given(corridor_like | st.integers().map(lambda n: f"AB-CD-{n}"))
def test_parse_transit_corridor_structured_no_crash(s):
    try:
        result = parse_transit_corridor(s)
    except Exception as e:
        assert False, f"parse_transit_corridor crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or isinstance(result, str)


@given(codes, codes, st.integers(min_value=1, max_value=100))
def test_parse_transit_corridor_valid(orig, dest, zones):
    result = parse_transit_corridor(f"  {orig}-{dest}-{zones} ")
    fare = 2.50 + zones * 0.75
    assert result == f"Route {orig}->{dest}: {zones} Zones, Fare ${fare:.2f}"


# ---------------------------------------------------------------------------
# lookup_concession_discount
# ---------------------------------------------------------------------------
@given(st.text() | st.none())
def test_lookup_concession_discount_safe(s):
    try:
        result = lookup_concession_discount(s)
    except Exception as e:
        assert False, f"lookup_concession_discount crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or result in VALID_CATEGORIES.values()
@given(st.sampled_from(sorted(VALID_CATEGORIES)), st.sampled_from([str.lower, str.upper, str.title]),
       st.sampled_from(["", " ", "\t"]))
```

```
def test_lookup_concession_discount_valid(category, case, pad):
    assert lookup_concession_discount(pad + case(category) + pad) == VALID_CATEGORIES[category]


# ---------------------------------------------------------------------------
# compute_passenger_density
# ---------------------------------------------------------------------------
@given(st.text() | st.none())
def test_compute_passenger_density_safe(s):
    try:
        result = compute_passenger_density(s)
    except Exception as e:
        assert False, f"compute_passenger_density crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or isinstance(result, float)


@settings(max_examples=300)
@given(ratio_like)
def test_compute_passenger_density_structured_safe(s):
    try:
        result = compute_passenger_density(s)
    except Exception as e:
        assert False, f"compute_passenger_density crashed on {s!r} with: {type(e).__name__}: {e}"
    # A density must be a real, non-negative number - never nan / inf / negative.
    assert result is None or (math.isfinite(result) and result >= 0), \
        f"invalid density {result!r} for input {s!r}"


@given(st.integers(min_value=0, max_value=10_000), st.floats(min_value=0.1, max_value=1000))
def test_compute_passenger_density_valid(passengers, km):
    assert compute_passenger_density(f"{passengers}/{km}") == round(passengers / km, 2)
```